

5 COMMON JAVASCRIPT MISTAKES
BEGINNERS MAKE (AND HOW TO FIX THEM)

1. Misusing var Instead of let or const

The Mistake

Back in the day, `var` was the go-to way to declare variables. But it has some quirky behavior — like hoisting and loose scoping — that trips up newbies.

For example:

```
for (var i = 0; i < 3; i++) {  
  console.log(i); // 0, 1, 2  
}  
console.log(i); // 3 (Wait, what?!)
```

You'd expect `i` to disappear outside the loop, but `var` doesn't respect block scope. It "leaks" out, causing unexpected bugs.

The Fix

Use `let` or `const` instead. They're block-scoped and safer:

```
for (let i = 0; i < 3; i++) {  
  console.log(i); // 0, 1, 2  
}  
console.log(i); // ReferenceError: i is not defined (Much better!)
```

- Use **let** when the variable will change.
- Use **const** for values that won't (like constants or fixed references).

2. Forgetting to Handle Async Code Properly

The Mistake

JavaScript's asynchronous nature (think API calls or timeouts) confuses beginners. A classic error is assuming code runs in order:

```
let data;  
fetch("https://api.example.com/data")  
  .then((response) => response.json())  
  .then((result) => (data = result));  
console.log(data); // undefined (Oops!)
```

The **console.log** runs before the **fetch** finishes because it's async.

The Fix

Use **async/await** to make async code feel synchronous:

```
async function getData() {  
  const response = await fetch("https://api.example.com/data");  
  const data = await response.json();  
  console.log(data); // Actual data!  
}  
getData();
```

Wrap it in a function, add **try/catch** for errors, and you're golden.

3. Mutating Arrays or Objects by Accident

The Mistake

JavaScript passes objects and arrays by reference, not value. Beginners often change the original data without meaning to:

```
const original = [1, 2, 3];  
const copy = original;  
copy.push(4);  
console.log(original); // [1, 2, 3, 4] (Wait, I only changed copy!)
```

Both **original** and **copy** point to the same array in memory.

The Fix

Create a real copy using the spread operator (...) or methods like **slice()**:

```
const original = [1, 2, 3];  
const copy = [...original]; // New array  
copy.push(4);  
console.log(original); // [1, 2, 3]  
console.log(copy); // [1, 2, 3, 4]
```

For objects, use **const copy = { ...original };**. Problem solved!

4. Overcomplicating Loops with Wrong Conditions

The Mistake

Loops are a beginner's rite of passage, but off-by-one errors or infinite loops are too common:

```
let numbers = [1, 2, 3];  
for (let i = 0; i <= numbers.length; i++) {  
  console.log(numbers[i]);  
}  
// Output: 1, 2, 3, undefined (Why undefined?)
```

The `<=` means it runs one extra time, accessing an index that doesn't exist.

The Fix

Use `<` instead of `<=`, or better yet, use modern methods like `forEach`:

```
let numbers = [1, 2, 3];
for (let i = 0; i < numbers.length; i++) {
  console.log(numbers[i]); // 1, 2, 3
}

// Or simpler:
numbers.forEach((num) => console.log(num)); // 1, 2, 3
```

`for...of` is another clean option: `for (const num of numbers) { console.log(num); }`.

5. Ignoring this Keyword Confusion

The Mistake

The `this` keyword in JavaScript changes depending on context, and beginners often lose track:

```
const obj = {  
  name: "Alex",  
  greet: function () {  
    console.log("Hi, " + this.name);  
  },  
};  
const sayHi = obj.greet;  
sayHi(); // "Hi, undefined" (Where'd Alex go?)
```

Here, **this** isn't bound to **obj** anymore — it's the global object (**window** in browsers).

The Fix

Use arrow functions (which inherit **this** from the surrounding scope) or bind the method:

```
// Option 1: Arrow function
const obj = {
  name: "Alex",
  greet: () => console.log("Hi, " + this.name), // Nope, still tricky with arrows!
  // Better:
  greet() {
    console.log("Hi, " + this.name);
  },
};
obj.greet(); // "Hi, Alex"

// Option 2: Bind it
const sayHi = obj.greet.bind(obj);
sayHi(); // "Hi, Alex"
```

For beginners, sticking to regular methods inside objects and avoiding loose function calls is safest.

What is Type Coercion in JavaScript?

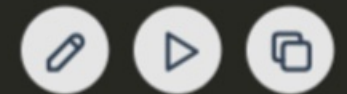
JavaScript is a loosely typed or dynamic language, which means it automatically converts types when necessary, making it easier for developers to work with different data types in expressions, comparisons, and operations. However, this flexibility can sometimes lead to unexpected results if you're not careful.

Type Coercion in JavaScript

In JavaScript, Type Coercion happens when JavaScript automatically changes one type of value into another. Sometimes, this automatic conversion can cause unexpected results if you're not aware of how it works.

- **Automatic Conversion:** When required, the JavaScript automatically converts required data types.
- **Works with Three Types:** String, Number, and Boolean coercion.
- **Can Lead to Unexpected Results:** If they are not handled properly, they may cause unintended bugs.
- **Implicitly Occurs:** Automatically converts the type of the value from one to another.

```
console.log(5 + "10");
```



Output:

"510"

In this example

- JavaScript converts the number 5 into a string.
- Concatenates it with "10".

How Type Coercion Works?

In JavaScript, type coercion mainly occurs in the three ways:

String Coercion

It occurs when the string is combined with the non-string using (+). JavaScript converts numbers and booleans into strings before concatenation.

```
console.log("5" + 2);  
console.log("5" + true);
```



Output:

```
"52"  
"5true"
```

- The number 2 is coerced to a string and then concatenated with the string “5”, resulting in “52”.
- The boolean true is coerced into the string “true”, and the two strings are concatenated.

Number Coercion

In the number coercion, JavaScript converts the string into a number before operating.

```
console.log("5" - 2);  
console.log("5" * 2);  
console.log("10" / "2");
```



Output

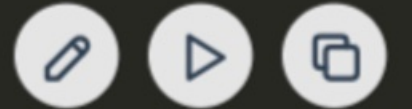
```
3  
10  
5
```

The string is coerced into a number before performing the arithmetic operation.

Number Coercion

In the number coercion, JavaScript converts the string into a number before operating.

```
console.log("5" - 2);  
console.log("5" * 2);  
console.log("10" / "2");
```



Output

```
3  
10  
5
```

The string is coerced into a number before performing the arithmetic operation.

Common Issues of Type Coercion

Comparing Different Data Types

Comparison Operator(==), allows coercion due to which the unexpected conversions occur. To avoid this, we should use the strict equality(===) operator.

```
console.log(0 == "0");  
console.log(0 == false);  
console.log(" " + 0 == 0);
```



Output

```
true  
true  
true
```

Boolean Coercion

JavaScript treats the true value as '1' and the false value as '0'.

```
console.log(Boolean("hello"));  
console.log(Boolean(0));  
console.log(Boolean([]));
```



Output

```
true  
false  
true
```

Non-empty strings are coerced to true, while 0 is coerced to false.

NaN Comparisons

NaN is not equal to itself, so checking with `isNaN()` is the best way to detect it.

```
console.log(NaN == NaN);  
console.log(isNaN(NaN));
```



Output

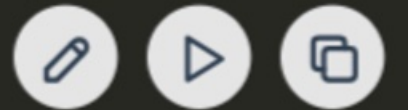
```
false  
true
```

In this example, NaN is not equal to itself, so the `isNaN()` function is the preferred way to check for NaN.

Operations on null and undefined

Null and undefined behave unexpectedly.

```
console.log(null == undefined);  
console.log(null === undefined);  
console.log(null + 1);
```



Output

```
true  
false  
1
```

Best Practices to Avoid Type Coercion Issues

Use === Instead of ==

When we use strict equality, instead of the comparison operator, it prevents unnecessary types of coercion.

```
console.log(5 === "5");
```



Output

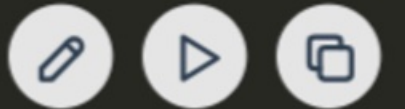
false

=== ensures no implicit type conversion occurs and both values must be of the same type.

Use Explicit Conversion

Explicit conversion converts the value manually due to which there are fewer chances of errors in the code.

```
console.log(Number("123"));
```



Output

123

This ensures that you're working with the correct type, reducing the chance of errors during operations.

Avoid False Value Confusion

Always check for null, undefined, or empty strings explicitly.

```
if (value !== null && value !== undefined) {  
    console.log("Value exists");  
}
```

This ensures that only non-null and defined values are considered valid.

Use parseInt() and parseFloat() for Number Conversion

```
console.log(parseInt("42px"));  
console.log(parseFloat("3.14abc"));
```



Output

```
42  
3.14
```

This will parse the number part of a string, ensuring a valid numeric conversion.

Handle NaN Properly

Use `isNaN()` to check if a value is NaN instead of comparing it directly.

```
if (isNaN(value)) {  
    console.log("Invalid number");  
}
```

This ensures you're correctly detecting NaN and handling it appropriately.



```
// How Hoisting Works  
// in JavaScript
```

What is Hoisting in JavaScript?

Hoisting is JavaScript's default behavior of moving declarations to the top of their scope before code execution. This means that variables, functions, and classes can be referenced in your code before they are defined, albeit with some caveats.

How Does Hoisting Work in JavaScript?

Hoisting applies differently to variables, functions, and classes. Here's a closer look:

Variable Hoisting

In JavaScript, variable declarations using `var`, `let`, and `const` are hoisted. However, the behavior differs depending on the keyword used.

Hoisting with var

Variables declared with `var` are hoisted and initialized to `undefined`:

```
console.log(myVar); // undefined
var myVar = 10;
console.log(myVar); // 10
```


Hoisting with let and const

Variables declared with `let` and `const` are hoisted but not initialized. Accessing them before the declaration results in a `ReferenceError` due to the Temporal Dead Zone (TDZ):

```
console.log(myLet); // ReferenceError
let myLet = 20;

console.log(myConst); // ReferenceError
const myConst = 30;
```

Function Hoisting

Function Declarations

Function declarations are fully hoisted, meaning they can be called before they appear in the code:

```
sayHello();  
  
function sayHello() {  
    console.log("Hello, world!");  
}
```

Function Expressions

Function expressions, including those defined using `var`, `let`, or `const`, are not hoisted in the same way:

```
sayHello(); // TypeError: sayHello is not a function
```

```
var sayHello = function () {  
  console.log("Hello, world!");  
};
```

Class Hoisting

Classes are hoisted in JavaScript, but like `let` and `const`, they remain in the Temporal Dead Zone until the declaration is executed:

```
const myInstance = new MyClass(); // ReferenceError
class MyClass {
  constructor() {
    console.log("Instance created");
  }
}
```


Common Pitfalls of Hoisting

Unintended Behavior with var

Variables declared with `var` can lead to unexpected bugs due to their initialization to `undefined`:

```
console.log(counter); // undefined  
var counter = 5;
```

FAQs About JavaScript Hoisting

1. What gets hoisted in JavaScript?

Variable and function declarations are hoisted. However, only function declarations and var variables are initialized during the hoisting process.

2. Are let and const hoisted?

Yes, but they remain in the Temporal Dead Zone until their declaration is executed.

3. How does hoisting affect arrow functions?

Arrow functions assigned to variables behave like function expressions and are not hoisted in the same way as function declarations.

Temporal Dead Zone Confusion

The Temporal Dead Zone can confuse developers unfamiliar with `let` and `const` behavior.

Always declare variables before using them to avoid `ReferenceErrors`.

Best Practices to Avoid Hoisting Issues

1. Use `let` and `const` Instead of `var`: Avoid using `var` to prevent unintentional behavior.
2. Declare Variables at the Top: Always declare variables at the top of their scope for better readability and to avoid confusion.
3. Understand the Scope of Variables: Familiarize yourself with block scope (`let`, `const`) vs. function scope (`var`).
4. Write Clear, Predictable Code: Avoid relying on hoisting behavior. Declare functions and variables explicitly before using them.

Conclusion

Hoisting is an important concept in JavaScript that influences how your code is interpreted. By understanding the nuances of hoisting and following best practices, you can write more reliable and maintainable code.

If you found this guide helpful, share it with your fellow developers and explore our blog for more JavaScript tips and tutorials.